Lab 2 — RWA Tokenization: A Short Walkthrough

This is a written walkthrough of the Lab 2 session, for students who want a narration to follow alongside the repository. We did not keep a recording of the session, so this document replaces it.

Keep the repo open while you read: `README.md`, `EXERCISES.md`, and `FOUNDRY-101.md` are the authoritative sources. This page adds the spoken thread that ties them together.

---

1. The one idea

In Lab 1, the collateral was already a token. The chain could check everything by itself: `totalCollateral() == totalSupply()` was a statement about two numbers that both lived on-chain.

In Lab 2, that assumption is gone. The asset is a short-dated US Treasury bill, held by a custodian, in a vault, off-chain. What you hold on-chain is **not the bill**. It is a **claim on the bill**.

So the subject of this lab is the machinery in between:

> **The bridge** — custody, attestation, physical redemption, and permissioning.

Four planks. Every exercise is built around one of them. If you catch yourself only reading NAV numbers and yields, you have drifted back into Lab 1. The interesting question is never "what is the price?" — it is "who says so, and what happens when they lie?"

---

2. The cast

Seven contracts, four planks, one cash token.

Contract	What it is	Plank
`MockUSDC`	The cash that crosses the bridge. 6 decimals, with a faucet	—
`ComplianceRegistry`	The guest list. `setWhitelisted()` / `isWhitelisted()`	admission
`TBillToken`	The claim itself. ERC-20, 18 decimals, gated on both endpoints of every transfer	admission
`TBillVault`	The on-ramp and the books. Prices subscriptions at the attested NAV	attestation
`MockPriceFeed`	The NAV oracle. 8 decimals; `1.00e8` means par	attestation
`MockTBillCustodian`	The custodian / SPV. Answers one question: `realHoldings()`	custody
`RedemptionQueue`	T+1 settlement. Shares in, a ticket out, cash later	redemption

Two decimal scales are worth memorising now: **USDC is 6 decimals, shares are 18, the NAV is 8.** Ex5 is where all three meet, and getting the arithmetic wrong there does not revert — it just pays the wrong amount.

---

3. Your first ten minutes

Run these in order and do not skip the first one:

```
make doctor        # environment check; all green before anything else
make test          # checkpoint: 7 passed — this is the answer key, it should be green
make exercise      # your task list — red right now, and that is correct
```

Then start a local chain and deploy:

```
make anvil         # terminal A — leave it running
make deploy-anvil  # terminal B — prints the admin and seven contracts
```

Export the addresses you need (`$USDC`, `$COMPLIANCE`, `$FEED`, `$TBILL`, `$CUSTODIAN`, `$VAULT`, `$QUEUE`). Anvil starts clean every time, so restarting means redeploying.

One rule for the whole lab: **red is the task, not a bug.** `make exercise` is red because the tests describe what you have not written yet. It turns green when you are done.

---

4. What each exercise is really asking

Ex0 — Environment

Get `make test` to 7 passed and `make doctor` to all green. Nothing else. If a check fails, the fix command is printed next to it.

Ex1 — Walk the bridge once (not graded)

Do this on the command line before touching any test. Fund yourself from the faucet, subscribe, look at the NAV, look at what the custodian claims to hold, then try to redeem.

The moment worth noticing: after `subscribe`, your share **balance** is fixed. When the NAV moves, the **value** of that balance moves. A stablecoin would have stayed 1:1. That difference is the whole difference between a pegged token and a claim.

`make redeem` will revert until Ex5 is done. That is expected — the queue does not exist yet.

Ex2 — Custody

You will prove two things: you can hold shares while the custodian holds nothing, and "the backing" is an integer that a permissioned address typed in.

Open `test/exercises/01_BridgeTasks.t.sol` and read both sides at once — your balance and `realHoldings()`. Pay attention to who is allowed to write that number.

Ex3 — Attestation

You will prove that one call re-prices the whole book. The share **count** does not change; the **value** does. Every holder is re-priced at once, because the vault reads one number.

Ask yourself while the test is green: what did the chain check that number against? The answer is the point of the exercise.

Ex4 — Admission

Three tests: no whitelist means no shares even if you pay; a transfer checks the receiver as well as the sender; and a frozen holder cannot be burned either.

That third one is the surprising one. The same guard that protects you from the issuer also stops the issuer from seizing your shares. Whether that is a feature or a bug is a discussion question, not a coding question.

Use the exact revert selector in your assertions. A vague "it reverted" stays green even after the guard is removed.

Ex5 — The redemption queue

Open `src/exercises/RedemptionQueue.sol` and fill in the four TODOs. This is the physical-redemption plank: a tokenized fund settles T+1, so redemption is a queue, not a swap.

Things to hold in mind:

- Three decimal scales meet here. `EXERCISES.md` gives the conversion; do not guess it.
- A ticket is all-or-nothing. A partial settlement carries the remainder forward.
- The queue must be whitelisted in the deployment, because burning its escrowed shares counts as a balance change. If you wire the system by hand and skip that, the whole loop reverts.

Ex6 — The two invariants

Invariant testing turns the usual test around: let the machine call operations in random order, then ask whether a property ever breaks.

Ex6 depends on Ex5. With the queue still stubbed, the fuzzer only sees reverts and finds nothing.

When an assertion fails, Foundry prints the counterexample sequence. Walk it by hand — that is the closest thing in this lab to a real audit. Then answer the question the code cannot: there is a third relation that looks like an invariant and is not. Write it up in `STUDENT-QUESTIONS.md`.

Ex7 — The challenge (optional)

You are a KYC'd customer who also holds the reporter's key. End with more USDC than you started with, at the fund's expense. The vault trusts one number for both pricing and payouts; the puzzle is to find the cheapest path from "I can write that number" to "I have cash I did not earn."

No hints beyond this. `make challenge` is red until you solve it.

---

5. How to work without getting stuck

1. Read the test function name. It is a sentence describing the property.
2. Read the assertion at the bottom. That is the acceptance criterion.
3. Run only that group: `forge test --match-test Ex3 -vvv`.

4. When a test reverts unexpectedly, read the revert reason before changing code.
5. If the machine-level details are unfamiliar — cheatcodes, `-vvv`, what a test skeleton looks like — read `FOUNDRY-101.md`. It is written for exactly this moment.

---

6. What you submit

Tier 1 is Ex0—Ex6. Ex1 is a warm-up and is not graded. Ex7 is optional.

Your submission is **one zip of the project**:

Keep `lib/` inside the archive — it lets the grader run `make test` offline.

The zip must contain:

1. the code, with `make exercise` green;
2. your answers in `STUDENT-QUESTIONS.md`;
3. an architecture diagram in `README.md` showing the **four planks** and where cash and shares cross the bridge.

Tier 2 (bonus) is the Sepolia deployment with all seven addresses and their Etherscan links listed under `## Deployment (Sepolia)` in your `README.md`.

---

7. The sentence to take away

A token is not the asset. It is a claim on the asset, and every claim has an issuer.

The chain can verify the claim. It cannot verify the asset. Everything in this lab lives in that gap.